SWASTHYASETU

AI-powered multilingual healthcare communication

Bridging language barriers between patients and doctors through speech AI, contextual translation and structured medical records.

Project type	Hackathon MVP / healthcare communication platform
Primary users	Doctors consulting patients who speak a different language
Initial languages	Hindi, Marathi and English, including supported code-mixed speech
Application	Doctor-operated Flutter mobile application

PROJECT DESCRIPTION & MVP REFERENCE

Prepared for planning, implementation and hackathon demonstration. This document records the currently agreed MVP scope; technical choices may be revised after API testing.

1. Project overview

SwasthyaSetu is a doctor-operated multilingual healthcare communication application designed to reduce language barriers during medical consultations. A doctor uses one phone to manage the consultation. The patient speaks in their selected language, while the application transcribes the conversation, translates speech when required, and presents the doctor with understandable information.

The application also converts the consultation into a structured draft medical record. The doctor reviews and approves the draft before it is saved. Follow-up appointment details can then be sent to the patient's mobile number by SMS.

Core value proposition

Help patients explain their concerns in a familiar language while giving doctors a practical way to understand, document and revisit the consultation.

2. Problem statement

Migrant workers and other patients may visit healthcare providers who do not speak their preferred language. This can make it difficult to describe symptoms, explain duration or medical history, understand a doctor's questions, and retain follow-up instructions. Generic translation can also mishandle code-mixed speech, medical terminology, medicine names, numbers and units.

SwasthyaSetu aims to address these issues through speech recognition, contextual translation, speech synthesis and doctor-reviewed record generation.

3. Target users and MVP boundaries

Area	MVP decision
Doctor accounts	One doctor account / one-doctor demonstration
Patient interaction	Doctor operates the application; patient only speaks
Device	One phone belonging to the doctor
Languages	Hindi, Marathi and English
Speech style	Code-mixed speech is in scope
Speaker identification	Doctor manually selects Patient or Doctor before each turn
Consultation mode	Real-time, turn-based speech exchange
Translation condition	Translation is used when patient and doctor language settings differ; same-language consultations can be transcribed and processed without translation

MVP assumption: the patient's selected language is treated as the primary language for the consultation. Automatic language switching is not a required MVP feature. Code-mixed speech within the supported language set should still be tested.

4. End-to-end user workflow

1. Doctor setup

- The doctor signs in and selects a preferred language: Hindi, Marathi or English.
- The language setting is saved and used as the default target language for the consultation.

2. Patient registration

- The doctor creates or finds a patient record.
- The doctor enters basic details such as name, age and mobile number, and selects the patient's preferred language.
- A unique patient ID links the patient's future consultations.

3. Consultation

- The doctor starts a consultation and selects the active speaker: Patient or Doctor.
- The phone captures the active speaker's speech. The interface shows the original transcription.
- Gnani speech-to-text (STT) transcribes the speech. If the two language settings differ, the OpenAI API translates the text into the other party's language.
- Gnani text-to-speech (TTS) can speak the translated response. The doctor and patient communicate through the same phone, taking turns.
- Both original and translated text are associated with the selected speaker and consultation.

4. Record generation and approval

- After the consultation, the OpenAI API extracts relevant information into a predefined structured format.
- The application marks the result as an AI-generated draft. It does not treat extracted diagnoses or prescriptions as confirmed.
- The doctor reviews, edits and approves the record. Only approved information is saved as the clinical record.

5. Follow-up

- The doctor selects or confirms the next visit date and time.
- The application sends a minimal SMS reminder containing appointment details and clinic contact information.
- Sensitive medical details and the full consultation record are not sent through ordinary SMS.

5. Consultation modes

Scenario	Expected behavior
Same language	Record and transcribe both sides, extract information and prepare a draft record. Translation can be skipped.
Different languages	Transcribe each speaker, translate to the other party's configured language, and use TTS to deliver the translated response.
Code-mixed speech	Attempt transcription while preserving the meaning of mixed Hindi, Marathi and English. Flag uncertain terms for review.

The MVP uses manual speaker selection rather than automatic speaker diarization. The doctor changes the active speaker before each turn.

6. Proposed technical architecture

Layer	Technology / responsibility
Mobile application	Flutter: doctor interface, patient registration, language settings, speaker selection, consultation screen, review and patient history.
Backend	Python FastAPI: APIs, consultation orchestration, audio handling, provider integrations, validation and persistence.
Speech-to-text	Gnani AI STT: transcription of patient and doctor speech; evaluate streaming support and code-mixed accuracy.
Translation and extraction	OpenAI API: contextual translation and structured extraction of consultation information.
Text-to-speech	Gnani AI TTS: spoken delivery of translated text in the recipient's language.
Database	PostgreSQL: patients, doctors, consultations, transcripts, extracted information, approvals and follow-ups.
Notifications	SMS provider: appointment reminders with minimal non-sensitive information.
Audio storage	Optional object storage for temporary or retained audio, subject to a defined retention policy.

High-level data flow

Step	Flow
1	Flutter app → FastAPI: speaker selection and audio stream / audio chunks
2	FastAPI → Gnani STT: speech recognition
3	FastAPI → OpenAI: translate transcript when required
4	FastAPI → Gnani TTS: synthesize translated text
5	FastAPI → Flutter: transcript, translation and playback response
6	FastAPI → OpenAI: extract structured consultation draft
7	Flutter → FastAPI: doctor edits and approval
8	FastAPI → PostgreSQL / SMS provider: save approved record and send reminder

Implementation note

For the first prototype, implement one speech turn at a time and validate the complete STT → translation → TTS loop. Add streaming or incremental delivery after the basic loop is reliable. Real-time performance depends on the Gnani API mode, network conditions, audio chunking and playback behavior.

Single-device audio constraint

Because the same phone records speech and plays synthesized audio, the microphone may capture the TTS output. Use headphones where practical, and evaluate echo cancellation, playback-aware microphone suppression or a short capture pause during TTS playback.

7. Structured patient records

The database should preserve a patient's longitudinal history. Each patient can have multiple consultations, and each consultation can contain speaker-labelled transcripts, extracted information, doctor edits and an approval state.

Entity / table	Example fields
doctors	doctor_id, name, credentials, preferred_language
patients	patient_id, name, age or date_of_birth, mobile_number, preferred_language
consultations	consultation_id, patient_id, doctor_id, date_time, language_pair, status
transcripts	transcript_id, consultation_id, speaker, original_text, translated_text, timestamp
symptoms	symptom_id, consultation_id, symptom, duration, severity, reported_by
medical_history	history_id, patient_id, condition, notes, source
diagnoses	diagnosis_id, consultation_id, diagnosis, confirmation_status, doctor_id
prescriptions	prescription_id, consultation_id, medicine, strength, dose, frequency, duration, approval_status
followups	followup_id, consultation_id, date_time, SMS_status

Example of AI-generated draft

Illustrative example only; not a medical recommendation.

Field	Draft value
Patient-reported symptoms	Fever; headache
Duration	Fever: 3 days; headache: 2 days
Medical history	Not stated in the conversation
Diagnosis	Not specified; doctor to enter or confirm
Prescription	Empty until entered and approved by doctor
Follow-up	Pending doctor input
Record status	Pending doctor review

Data integrity rules

- Distinguish patient-reported symptoms from doctor-confirmed diagnoses.
- Do not infer missing values. Use null, 'not stated' or 'needs review'.
- Preserve medicine names, strengths, quantities, dosage units and frequencies; flag discrepancies.
- The AI must not independently create or alter a prescription or diagnosis.
- Store original transcript and translated text separately so that translation can be audited.

8. Reliability, privacy and safety requirements

- Use simulated or fictional patient data for the hackathon demonstration.
- For real-world use, obtain appropriate patient consent before recording or processing consultations. Omitting a consent-management feature from the MVP does not remove this requirement.
- Protect patient data with authentication, role-based access, encrypted network connections and appropriate database/storage safeguards.
- Keep API keys on the backend; never embed OpenAI or Gnani credentials in the Flutter application.
- Define audio retention and deletion rules. Avoid keeping raw audio longer than necessary.
- Show transcription and translation uncertainty. Allow the doctor to correct mistakes.
- Keep a clear distinction between AI-generated drafts and doctor-approved records.
- SMS should contain only necessary appointment information, not diagnoses, prescriptions or detailed medical history.
- This MVP is a communication and documentation aid, not an autonomous diagnostic or treatment system.

9. MVP feature checklist

Feature	MVP status
Doctor language settings	In scope
Patient registration and language selection	In scope
Hindi, Marathi and English	In scope
Code-mixed speech support	In scope; validate with test utterances
Manual Patient / Doctor speaker selection	In scope
Single-phone audio capture	In scope
Gnani STT	In scope
OpenAI contextual translation	In scope when languages differ
Gnani TTS	In scope
Consultation transcript	In scope
AI-generated structured draft	In scope
Doctor edit and approval	In scope
Patient history and consultation retrieval	In scope
Doctor-selected follow-up and SMS	In scope
Automatic language switching	Out of scope
Multiple doctors / multi-clinic support	Out of scope
Autonomous diagnosis or prescription	Out of scope
Offline mode and hospital integrations	Out of scope

10. Suggested development sequence

Phase	Deliverable
1. Foundation	FastAPI project, PostgreSQL schema, Flutter navigation and doctor settings.
2. Patient management	Patient registration, patient search and consultation creation.
3. Speech pipeline	Gnani STT integration, audio capture and manual speaker selection.
4. Translation loop	OpenAI translation and Gnani TTS playback for the three supported languages.
5. Record generation	Structured extraction, schema validation and draft review screen.
6. Persistence	Doctor approval, patient history and consultation retrieval.
7. Follow-up	Doctor-selected follow-up date and SMS integration.
8. Demo hardening	Latency tests, noisy audio tests, code-mixed test cases and failure handling.

11. Testing and success criteria

The MVP should be evaluated with scripted, fictional consultations covering the supported languages and realistic code-mixing. Testing should include both same-language and different-language scenarios.

Test area	What to verify
Transcription	Names, symptom phrases, numbers, units and code-mixed utterances are transcribed accurately.
Translation	Medical meaning is preserved; ambiguous terms are flagged rather than guessed.
Speaker attribution	Each transcript segment is associated with the manually selected speaker.
Speech playback	TTS output is understandable and does not cause severe microphone echo.
Latency	Measure time from end of a speaker's turn to translated text and audio playback.
Extraction	Symptoms, duration and history are extracted without inventing missing information.
Clinical review	Doctor can edit and approve the draft before it becomes a saved clinical record.
Follow-up	Correct appointment details reach the intended test phone number.

Recommended demo scenario

A fictional patient selects Hindi and reports: “Mujhe three days se fever hai aur headache bhi hai.” The doctor uses English. The application transcribes the patient's code-mixed statement, translates it for the doctor, and supports the doctor's English response being translated and spoken back in Hindi. At the end, the system drafts a record showing patient-reported fever and headache with their stated durations. The doctor reviews and approves the record, selects a follow-up date, and triggers an SMS appointment reminder.

12. Risks and mitigations

Risk	Mitigation
Medical mistranslation	Preserve original text, validate entities and numbers, flag uncertainty, doctor review.
Incorrect extraction	Constrained schema, null for unknown values, draft status and doctor approval.
Noisy clinic audio	Test microphones and environments; allow repeat / correction; consider noise suppression.
TTS feedback into microphone	Headphones, echo cancellation or pause capture during playback.
Network/API failure	Show clear status, allow retry and preserve unsent transcript segments safely.
High latency	Measure each pipeline stage; use streaming where supported and avoid unnecessary model calls.
Sensitive data exposure	Backend-only API keys, access controls, encryption, limited retention and minimal SMS content.

13. Future scope (after MVP)

- Support additional Indian languages and more robust automatic language identification.

- Multi-doctor and multi-clinic accounts with controlled record sharing.
- Patient-facing app or QR-based record access with patient authorization.
- Prescription explanation in the patient's language, based only on doctor-approved instructions.
- Offline-friendly patient history and deferred synchronization.
- Integration with healthcare record standards and authorized clinical systems.
- Improved speaker diarization and hands-free conversation mode.

Project positioning

SwasthyaSetu is a multilingual healthcare communication and documentation assistant. It combines Gnani speech recognition and speech synthesis with OpenAI-powered contextual translation and structured information extraction, while keeping the doctor responsible for reviewing and approving clinical records.

Document status: MVP reference specification based on the currently agreed project decisions. Provider model names, API capabilities, pricing and streaming behavior should be verified against current official documentation during implementation.